# CS2107 Cheatsheet

Anselm Long AY25/26
Adapted from benluiwj

## CIA Triad

**Confidentiality** - Asset viewed only by authorised parties; prevent unauthorised disclosure.
**Integrity** - Asset modified only by authorised parties; prevent unauthorised modification.
**Availability** - Asset used by any authorised party; prevent unauthorised withholding.

## Encryption

Requirements:
- **Correct** ($\text{DEC}_k(\text{ENC}_k(p)) = p$),
- **Secure** Ciphertext should be indistinguishable from a random stream (difficult to get $k/p$ from $c$)
- **Efficient**.

Alice: originator; Bob: receiver; Eve: sniff; Mallory: sniff and spoof.
**Deterministic** - same plaintext + key ⇒ same ciphertext.
**Probabilistic** - different ciphertexts even with same key and plaintext.
**Encryption guarantees Confidentiality only!**
**Symmetric-Key** - same key for enc/dec.
**Public-Key** - public key encrypts, private key decrypts.
**Kerckhoffs' Principle** - System must be secure even if everything except the secret key is public knowledge.
**Compression** on encrypted file yields little gain (encrypted file is already random).

## Attack Models

- **Ciphertext-only** - given ciphertexts, may know some plaintext properties
- **Known-plaintext** - given ciphertexts and corresponding plaintexts
- **Chosen-plaintext** - access to encryption oracle (choose any plaintext, get ciphertext)
- **Chosen-ciphertext** - access to decryption oracle (choose ciphertext, get plaintext)

In PKC, encryption oracle always available (public key is known) ⇒ chosen-plaintext must always be considered. Padding oracle is a weaker form of decryption oracle.
**Total break** - find the key. **Partial break** - recover plaintext or coarse info without finding key. **Distinguishability** - correctly distinguish ciphertexts of a given plaintext from another ciphertext of a different plaintext. If $S_1$ prevents more attacks than $S_2$, then $S_2$ is more secure w.r.t. the attack model.

## Ciphers

**Key Space** - set of all possible keys. **Key Space size** - number of keys. **Key size** - bits of one key.
**Diffusion** - change in plaintext affects many parts of ciphertext; information spread over entire ciphertext.
**Confusion** - attacker cannot predict what happens to ciphertext when one character in plaintext/key changes.

### Substitution Cipher

Key is a 1-to-1 substitution table. Monoalphabetic Cipher Key space size: 27!; key size: $\log_2(27!) \approx 94$ bits.
**Known-plaintext** - can obtain parts of key; fully determinable with enough pairs. **Known-ciphertext** - use **frequency analysis** if plaintext is English.
**Monoalphabetic** - fixed substitution per letter; vulnerable to frequency analysis on long ciphertexts.

### Permutation Cipher

Group plaintext into blocks of $t$ characters; apply permutation (shuffle) to each block. $t$ may be secret. **Known-plaintext** - very easy to determine key.
**Known-ciphertext** - easily broken for English.

### One Time Pad (OTP)

XOR: $A \oplus B = B \oplus A$, $A \oplus (B \oplus C) = (A \oplus B) \oplus C$, $A \oplus 0 = A$, $A \oplus A = 0$
**Enc:** plaintext ⊕ key → ciphertext. **Dec:** ciphertext ⊕ key → plaintext.
Exhaustive search does not work on OTP.
**Perfect secrecy**: attacker gains no additional information from ciphertext (except length), even with arbitrary runtime.

### Stream Cipher & IV

Without IV, encrypting $x, y$ with same key leaks $x \oplus y = (x \oplus k) \oplus (y \oplus k)$. IV makes encryption **non-deterministic**; different IVs ⇒ different ciphertexts for same plaintext. Generate a keystream and XOR with plaintext to get ciphertext. Short secret key → generator (e.g. RC4) → pseudorandom keystream matching plaintext length → XOR (OTP). Final ciphertext = IV ‖ OTP output. To decrypt: extract IV, regenerate keystream, XOR.

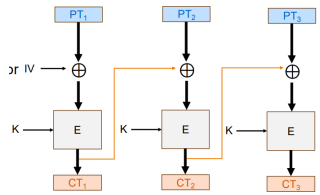
| | Stream | Block |
|---|---|---|
| **Pros** | Speed; low error propagation | High diffusion; immunity to insertion |
| **Cons** | Low diffusion; susceptibility to malicious insertions/modifications | Slowness; padding; error propagation |

## Block Cipher

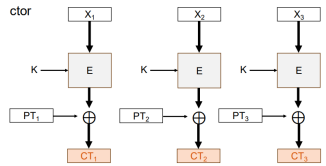
**Block size** - bits per plaintext block. **Key size** - bits of key (longer ⇒ more secure but slower).
**Data Encryption Standard (DES)** - symmetric key block cipher → block 64 bits, key 56 bits.
**Advanced Encryption Standard (AES)**: block 128 bits, key 128/192/256 bits.

### Modes of Operation

Extending encryption from a single block to multiple blocks to encrypt.
**Electronic Code Block (ECB)** - divide into blocks, encrypt each with same key. *Problem*: identical plaintext blocks ⇒ identical ciphertext blocks (leaks information).
**Cipher Block Chaining (CBC)** - each plaintext block XORed with previous ciphertext block before encryption.
- Encryption *cannot* run in parallel (depends on previous ciphertext block)
- Decryption *can* run in parallel (all ciphertext blocks available upfront)
- If first ciphertext block removed: first plaintext unrecoverable, rest OK
- **Vulnerable to Padding Oracle Attack**




**Counter Mode (CTR)** - turns block cipher into stream cipher; treats each block as a stream. **Vulnerable when counter is sequential.**




## Meet-in-the-Middle Attack

**2DES**: encrypt with DES twice using different keys. DES does not form a group ($E_{k_1}(E_{k_2}(x)) \neq E_{k_3}(x)$). Key length = 2 × 56 bits.
**Meet-in-the-middle** (known-plaintext): given $p$ and $c$, find two keys.
1. $C$ = encrypt $p$ with all possible keys
2. $P$ = decrypt $c$ with all possible keys
3. Find common element: $\text{DEC}_{k_1}(c) = \text{ENC}_{k_2}(p)$
4. Cost: $2^k + 2^k = 2^{k+1}$ ops; $\sim 2^k$ storage

**3DES**: $E_{k_1}(D_{k_2}(E_{k_3}(x)))$. With $k_1 = k_2$: same as single DES.
**Improvement with hashing**: store $k_1(m)$ in hash table, perform $k_2$(hash); total ≈ half of naive.

## Padding Oracle Attack

**Attacker has**: ciphertext, IV, access to padding oracle; no knowledge of secret key. **Goal**: recover plaintext.
**Padding oracle**: given ciphertext, outputs YES if padded correctly, NO otherwise.
**PKCS#7**: padding bytes are $0X \times X$. If last block full, add extra block of all zeroes.
**AES-CBC not secure against padding oracle attack** when used with PKCS#7.
**Algorithm**: modify IV byte-by-byte to IV′ such that oracle returns YES. Since $c$ unchanged, intermediate decryption block unchanged:

$$I = \text{IV}' \oplus x' = \text{IV} \oplus x \implies x = 04 \oplus \text{IV}'$$

**Getting padded bytes**: linear probe each byte of IV left-to-right; if oracle says NO, padding length $= 2^n + 1 - i$.
**Prevention**: deny access to oracle; change padding standard.

### Attacks on Implementation

IV must be **unpredictable** in AES-CBC. Use `java.security.SecureRandom` (not `java.util.Random`). Never design your own cryptosystem or make slight modifications to existing schemes.
**Side Channel Attack** - attack based on extra information from *implementation*, not design flaws.

## Public-Key Cryptography

Alice keeps private key secret, publishes public key. Encryption oracle always available. **Chosen-plaintext attack must always be considered in PKC.**
Security: given public key + ciphertext (not private key), difficult to find plaintext or derive private key.
Keys required: Symmetric: $\frac{n(n-1)}{2}$ pairwise keys. PKC: $n$ private + $n$ public; no prior meeting needed.

## RSA

**Key Generation**:
1. Choose large primes $p, q$ (secret); $n = pq$ (public)
2. Choose $e$ s.t. $\gcd(e, \phi(n)) = 1$, where $\phi(n) = (p-1)(q-1)$
3. Compute $d = e^{-1} \pmod{\phi(n)}$, i.e. $de \equiv 1 \pmod{\phi(n)}$
4. Public key: $(n, e)$. Private key: $(n, d)$

**Enc**: $c = m^e \pmod n$. **Dec**: $m = c^d \pmod n$ (since $(m^e)^d = m \pmod n$ for $m < n$).
**Special property**: can also use $d$ to encrypt and $e$ to decrypt (basis of signatures).
Usually $e = 65537$; $d$ via extended Euclidean algorithm.
**Security**: deriving private key from public key is as hard as factoring $n$.
Factoring possible by quantum computer ⇒ RSA broken by QC.
**Homomorphic**: $E(m_1 \cdot m_2)$ computable from $E(m_1)$, $E(m_2)$. Leads to attacks; use PKCS#1 padding.
**RSA slower than AES**: for large files, use hybrid: encrypt file with AES key $k \to C$, encrypt $k$ with RSA → $y$, send $(y, C)$.
**ECB with RSA**: independently encrypting plaintext pieces leaks information.

## Data Integrity (Hash)

Hash: arbitrary-length input → fixed-size digest. Key-less.
**Requirements**:
- **Efficient**: easy to compute $h(m)$
- **Pre-image resistant (one-way)**: given $y$, infeasible to find $m$ s.t. $h(m) = y$
- **Collision resistant**: infeasible to find distinct $m_1, m_2$ with $h(m_1) = h(m_2)$

Collision resistance ⇒ pre-image resistance (not the other way). **Do not use MD5**. Do not hash IV (known by attacker ⇒ insecure).
**Unkeyed hash use**: assume secure channel delivers $h(F)$. Obtain $F'$; if $h(F') = h(F)$, likely same file.

## Birthday Attack on Hash

Given $M$ messages tagged with random values in $\{1, \ldots, T\}$:

$$\text{If } M > 1.17\,T^{0.5} \implies \text{collision with probability} > 0.5$$

For $x$-bit digest: $T = 2^x$. Attacker needs $M \approx 2^{1+x/2}$ messages. Collision probability $\approx 1 - e^{-M^2/(2T)}$.
Digest must be sufficiently long (e.g. SHA-256 for 128-bit security) to resist birthday attacks.

## Data Authenticity (MAC & Signatures)

**Message Authentication Code (MAC)** - keyed-hash: arbitrary input + secret key → fixed-size tag. Secret key is shared, so guarantees authenticity. Only generated by someone who knows the key. **Security**: difficult to forge MAC of unseen message (existential forgery).
**CBC-MAC**: based on AES under CBC. **HMAC**: based on any iterative hash. HMAC faster than signatures. **AES-GCM** for authenticated encryption.

### Symmetric Key Setting

Use MAC for authenticity. MAC appended to data. Attacker may modify but detectable with high probability.

### Digital Signature (Public-Key Setting)

Owner signs with private key; anyone verifies with public key. **Security**: without private key, difficult to forge $s$.
**Generation**: $h(F)$ via unkeyed hash → sign with private key → signature $s$.
**Verification**: decrypt $s$ with public key → compare with $h(F)$.
**Non-repudiation**: signer cannot deny previous actions.
**RSA-based Signature**: private key encrypts hash → signature; public key decrypts for comparison.

## Password Authentication

**Entity Authentication** - connection-oriented; passwords, challenge-response, biometrics.
**Data-origin Authentication** - connectionless; implies data integrity; uses MAC or digital signature.
**Data-origin authenticity** ⇒ **data integrity** (not the other way).
Authenticity is stronger.
**Password** - weak auth: subject to replay attack (sniffed info can impersonate later).
**Strong authentication** - sniffed information cannot be used to impersonate.
**Bootstrapping** - establish common password. Attacker can intercept during bootstrap or use default.
**Password Reset** - only authorised entity can reset. Options: additional documents, security questions, recovery email (OTP link).

# Attacks on Password Systems

- **Online guessing** - directly interact with auth system
- **Offline guessing** - obtain password file
- **Dictionary attack** - words from dictionary, known compromised passwords
- **Shoulder surfing** - look-over-the-shoulder
- **Keylogger** - capture keystrokes, send to attacker
- **Spoofing** - fake login screen; defence: secure attention key
- **Phishing** - trick user to send password (emails, websites)
- **Spear Phishing** - targeted phishing
- **Vishing** - voice calls. **Smishing** - SMS phishing
- **Password Caching** - shared workstation caches keyed-in info
- **Insider Attack** - malicious sysadmin steals password file

## Preventive Measures

**Phishing**: user education, drills, phishing filtering (firewall, mail server, browser).
**Entropy** $H = L \log_2(N)$ bits for a password of length $L$ from a set of $N$ equally likely symbols. 94 for symbol set of lower, upper, number and symbols. Online attacks: $\geq 29$ bits (RFC 4086). Offline + crypto keys: $\geq 128$ bits (NIST) or 96 bits (RFC 4086).
**Password protection**: limit login attempts (delays, security questions, auto-lock), password checker, metering, ageing, usage policy.

## Protecting Password File

Store hash(salt‖password). Same password ⇒ different hash per user (due to salt).

# Multi-Factor Authentication (MFA)

At least 2 factors: **what you know** (password, PIN), **what you have** (card, OTP token), **who you are** (biometrics).
**OTP Token** - hardware generating one-time passwords; token and server share a secret. **Time-based**: from shared secret + current time. **Sequence-based**: event triggers password change.
**Password + SMS**: **SMS is not secure!** OTP only: more secure if adversary gets mobile (no transaction details leaked). OTP + details: more secure against malware on PC showing different transaction.
**Smartcard + Fingerprint**: insert smartcard → fingerprint verification. Template not stored in reader; reader communicates with server via secure channel.

# Token vs Session-ID Auth

**Token**: browser auto-authenticates after login. Has expiry; forced re-auth when expired.
**Session-ID**: server maintains userid ↔ SID database.

- On login: server generates unique, hard-to-guess SID, sends to user
- Subsequent requests: present SID; server verifies
- Update SID on session expiry or password change (logs out all devices)
- No cryptographic primitive required; needs database; slower to verify

# Public Key Distribution

**Public Announcement**: broadcast public key (email/website). Not standardised; no systematic verification.
**Publicly-Available Directory**: query server for name/key pairs. Server is bottleneck & single point of failure; must be online; cannot verify correctness of postings.

# PKI

**CA (Certificate Authority)**: issues and signs certs; has own key pair. CA public key assumed securely distributed.
Overcomes: (1) bottleneck of directory server, (2) verifier needing to be online at verification time (receiver carries the cert).
**Certificate** contains: identity of owner, public key of owner, validity window, CA's signature.
Receiver verifies CA signature ⇒ no one except CA can produce valid signature. CA also verifies info is correct.
**Self-Signed Certificate**: signed by entity's own private key. Used by root CAs or developers.
**Validation levels**: Domain (DV), Organisation (OV), Extended (EV, includes manual checks).

## Types of CA

Root CA: self-signed. Intermediate CA: Tier 1, 2,... Leaf CA: issues certs to end entities. Tree structure.
**Hierarchy of Trust**: leaf → intermediate → root.
**Certification Chain**: end-entity cert → intermediate CA certs → self-signed root CA (trust anchor). Each cert's issuer = subject of next cert in chain.

## Certificate Revocation

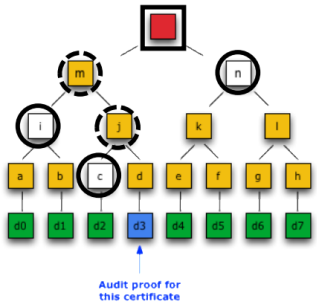
Reasons: private key compromised, issuing CA compromised, entity left org, business closed.
**Certficate Revocation List (CRL)**: CA periodically signs and publishes revocation list.
**Online Certificate Status Protocol (OCSP)**: query if certificate is still valid.
Risks of accepting expired certs: previous owner retains private key; could run spoofed HTTPS site.

## Limitations of PKI

**Compromised CA**: rogue CA can forge any certificate.
**Compelled Certificates**: governments can compel CAs to issue false certificates
**Null-byte injection**: browsers ignore substrings after null char when displaying but include when verifying certs.
**Typosquatting**: phishing site with domain differing by one character.

## Certificate Transparency

**Certificate Log** - public, verifiable, append-only log of certificates
**Merkle Tree** - leaves are hashes of individual certificates that have been appended to the log, nodes are hashes of paired child leaves or paired child nodes. Tree head is root, summary of all certificates. If any node is changed, root changes.




**Signed Certificate Timestamp (SCT)** - CA sends certificate to log server which issues SCT that it will insert within a Maximum Merge Delay (MMD).

# Strong Authentication

## Symmetric Challenge-Response

Shared secret key $k$:
1. Alice → Bob: "I am Alice"
2. Bob → Alice: $\text{ENC}_k(m)$ for random $m$
3. Alice → Bob: $m$ (after decrypting)
4. Bob verifies

Cannot replay (fresh random $m$ each time). **Unilateral**: only authenticates Alice.

## PKC Challenge-Response

1. Alice → Bob: "I am Alice"
2. Bob → Alice: random nonce $r$
3. Alice → Bob: $\text{Sign}_{k_{priv}}(r)$ + certificate
4. Bob: verify cert, extract public key, verify signature

Cannot derive private key; $r$ is fresh (nonce). If Mallory interrupts, introduce session key $k$ via key exchange.
**Mutual auth**: both need each other's public key. **Unilateral**: one side only.

# Key Exchange

## Unauthenticated Key Exchange

Goal: establish shared key (confidentiality only).
**Using PKC**: Alice generates keypair, sends $k_p$ to Bob. Bob encrypts random secret $s$ with $k_p$, sends ciphertext. Alice decrypts to get $s$ (shared key).
**Diffie-Hellman**: public parameters $g$ (generator), $p$ (large prime).

- Alice: choose $a$, send $g^a \bmod p$
- Bob: choose $b$, send $g^b \bmod p$
- Shared key: $g^{ab} \bmod p$

Based on Discrete Log Problem. **Vulnerable to MitM**: Mallory uses own parameters, decrypts and re-encrypts messages.

## Authenticated Key Exchange

**Station-to-Station Protocol**: DH + added signatures. Alice and/or Bob sign their DH values. Prevents MitM.

## Perfect Forward Secrecy (PFS)

If long term private key is compromised, past communication sessions still remain secure. Discard session keys after used to encrypt and decrypt. **DHKE** provides PFS, but **PKC** does not.

# Secure Channel

**Sequence numbers**: prevent reordering, detect duplicates, detect dropped records.
After authenticated key exchange, establish session keys $k$ (encryption) and $t$ (MAC):

$$\text{Data for } m_i: \quad E_k(i \| m_i) \| \text{MAC}_t(E_k(i \| m_i)) \quad \text{(Encrypt-then-MAC)}$$

# HTTPS & TLS/SSL

HTTPS = HTTP + SSL. TLS/SSL sits between transport and application layers.
**TLS involves**: cipher negotiation, authenticated key exchange, symmetric-key secure communication, optional renegotiation.
TLS **handshake is unencrypted**. MitM can see handshake but cannot derive session key.
**TLS Handshake**:
1. User requests certificate
2. Server sends cert
3. User verifies cert, extracts public key
4. User generates session key pair, encrypts with public key, sends to server
5. Session keys expire; renegotiation needed periodically

Asymmetric enc for key exchange; symmetric enc for messages. TLS is mutual but each stage is unilateral.

## TLS Renegotiation Attack

Attacker completes first TLS handshake, establishes session keys. Hijacks client connection, proxies client traffic over attacker's channel. Server treats this as a renegotiation. **Breaches integrity; does not compromise confidentiality.** Naive fixes fail: handshake numbers can be reordered by attacker. **Transport layer fix**: server signs message with private key, or MAC with previously established key.

# QUIC

Uses UDP for transport but integrates TLS directly into the protocol. Only requires 1 RTT for new connection compared to 3 RTT for TLS over TCP.

# Authenticated Encryption

Symmetric encryption returning ciphertext + authentication tag. Combines cipher and MAC.
$\text{AE}(K, M) = (C, T)$ and $\text{AD}(K, C, T) = M$ only if $T$ valid.

- **Encrypt-and-MAC**: compute cipher and MAC separately, send both. (SSH) MAC may leak info.
- **MAC-then-Encrypt**: compute MAC, append to plaintext, encrypt all. (SSL/TLS v1.2) Decryption needed on corrupted messages.
- **Encrypt-then-MAC**: encrypt first, then MAC the ciphertext. (IPsec) **Best**: no decryption on corrupted messages.

# Network Layers

**Packet Switching**: messages broken into packets, routed via switches/routers.
**Network Layering**: peer entities at same layer communicate via protocol; layer below provides services.
Layer-$N$ PDU = encapsulation of upper $(N+1)$ layer PDU. Intermediate nodes may modify headers.
**UDP**: connectionless, unreliable; packets may be lost.
**TCP**: connection-oriented (3-way handshake); reliable (retransmit, reorder, ACK). **Reliable ≠ secure**: intermediate nodes can read/modify headers and payload; vulnerable to MitM.

# Cryptographic Communication Channel

Protocol at layer $k$ protects layers $\geq k$ against attacker at layer $\leq k-1$.
**SSL/TLS**: on top of transport. End-to-end encryption. Link encryption: encrypts at each hop.
**WPA2**: protection at L2 (link) and L1 (physical). Attacker can learn MAC address.
**IPsec**: integrity/authenticity for IP (L3) but not confidentiality. Attacker cannot spoof but can sniff.